

BATTIFY

On-Demand Battery Delivery, Installation and Buyback Platform

Vyomesh Joshi, CSED
Ph No: 6284329484
Roll No: 1024030181
Email: `vjoshi_be24@thapar.edu`

Sarika Garg, CSED
Ph No: 6284532323
Roll No: 1024030741
Email: `sgarg1_be24@thapar.edu`

Shivinder Singh, CSED
Ph No: 9653797001
Roll No: 1024030213
Email: `ssingh25_be24@thapar.edu`

Yatish Goyal, CSED
Ph No: 9888327011
Roll No: 1024030187
Email: `ygoyal_be24@thapar.edu`

August 20, 2026

Project Proposal for UCS503P – Software Engineering Lab

Thapar Institute of Engineering and Technology

1 Higher-order Goal

Real-world impact

Battify aims to make battery replacement and servicing a fast, reliable and transparent service rather than a conventional retail purchase. The platform targets households, farmers, vehicle owners and small businesses that may need a replacement battery urgently but currently have to locate a retailer, arrange transportation, wait for availability and separately handle installation and disposal of the old battery.

The engineering objective is to build a software system that coordinates customers, inventory, delivery personnel, installation and old-battery buyback through one workflow. The project therefore has a direct real-world use case while providing enough software complexity to demonstrate the learning outcomes of UCS503.

2 Problem Statement

Battery failure is often time-sensitive. A failed inverter battery can leave a household without backup power, while a failed tractor or agricultural pump battery can interrupt farm operations. Existing purchasing methods commonly separate the activities of finding a battery, checking availability, transporting it, installing it and disposing of the old unit.

The major problems are:

- **Fragmented service:** battery purchase, delivery, installation and old-battery disposal are handled separately.
- **Uncertain availability:** customers may not know whether the required battery is actually in stock nearby.
- **Slow replacement:** urgent customers have limited options for same-day delivery and installation.
- **Poor workflow visibility:** customers often cannot track whether an order has been accepted, dispatched or installed.
- **Seasonal demand:** agricultural batteries experience demand spikes around sowing and harvesting periods, making inventory planning important.
- **Battery waste:** customers need a convenient mechanism for returning old batteries through a responsible recovery channel.

3 Proposed Solution

Software Proposition

Battify will be developed as a web-based platform connecting customers, inventory managers and delivery/installation agents.

The proposed system will provide:

- **Battery catalogue:** customers can select battery type, capacity, vehicle/application and preferred brand.
- **Location-aware ordering:** the system records the delivery location and identifies suitable nearby inventory.
- **Inventory management:** administrators can monitor stock, reserved units, low-stock products and seasonal demand.
- **Delivery and installation workflow:** orders move through defined states from placement to installation.
- **Old-battery buyback:** the old battery is recorded and its estimated buyback value is deducted from the order.
- **Customer tracking:** customers can see order status and installation progress.
- **Operations dashboard:** administrators can monitor orders, delivery times, stock levels and unresolved jobs.

4 Core Workflow

The principal workflow is:

1. Customer selects the required battery and provides vehicle or application details.
2. The system validates the request and checks inventory.
3. The customer selects a delivery location and preferred service slot.
4. An order is created and assigned an order ID.

5. The operations dashboard assigns the order to a delivery/installation agent.
6. The agent collects or verifies the old battery, delivers the new battery and performs installation.
7. The old battery is recorded for buyback/recycling.
8. The order is marked completed and the customer receives a final status and invoice.

4.1 Order State Model

A controlled state machine can be used:

Placed → Confirmed → Assigned → Dispatched → Delivered → Installed → Completed

Exceptional states such as *Cancelled*, *Out of Stock* and *Rescheduled* will also be supported.

5 Why This is Suitable for UCS503

Battify is intentionally designed as more than a simple e-commerce website. The core challenge is the coordination of multiple actors, inventory, geographical service, order states, installation and reverse logistics.

This creates a practical software engineering problem involving:

- requirements elicitation and validation;
- UML and workflow modelling;
- modular system and database design;
- architecture and API design;
- software testing and verification;
- Agile backlog and sprint management;
- energy-aware software and UI decisions;
- project scope, time and cost estimation.

6 Mapping to Course Learning Outcomes

CLO	How Battify Demonstrates the CLO
CLO 1	A software process model can be selected and justified for the project. An incremental/Agile approach is suitable because the system can be delivered in modules such as catalogue, ordering, inventory, dispatch and buyback. The project can compare Waterfall, Incremental and Agile approaches before selecting the final process.
CLO 2	Requirements can be elicited from multiple stakeholders: customers, administrators, inventory managers, delivery agents and installation personnel. Use-case, activity, sequence, class and state diagrams can model the resulting requirements, followed by a formal SRS.

CLO 3	The platform naturally supports separation of concerns between customer UI, operations dashboard, order management, inventory, dispatch and buyback. Database design, API design, MVC architecture, modularity, coupling and cohesion can all be demonstrated.
CLO 4	The order lifecycle provides clear testable states and business rules. Unit tests can cover price calculation, inventory reservation and buyback calculation; integration tests can cover order-to-dispatch workflows; system tests can cover complete customer journeys. Black-box and white-box testing strategies can also be documented.
CLO 5	The project can be managed through Agile user stories, product backlog, sprint backlog, story points, velocity and sprint reviews. UI sketches and story cards can be used before implementation, while Test Driven Development can be applied to critical modules.

7 Requirements Engineering

7.1 Stakeholders

The principal stakeholders are:

- Customer
- Delivery and installation agent
- Inventory manager
- Operations administrator
- Battery supplier
- Buyback/recycling partner

7.2 Representative Functional Requirements

1. The system shall allow customers to browse battery products.
2. The system shall allow customers to submit a replacement request.
3. The system shall check inventory before confirming an order.
4. The system shall reserve inventory for confirmed orders.
5. The system shall assign an eligible service agent to an order.
6. The system shall maintain an auditable order-status history.
7. The system shall calculate the final amount after applicable buyback credit.
8. The system shall allow administrators to update inventory.
9. The system shall provide customers with order tracking.
10. The system shall record completed installations and old-battery collection.

7.3 Representative Non-functional Requirements

- **Performance:** common customer operations should respond quickly under expected project-scale load.
- **Usability:** the customer workflow should require minimal steps and work well on mobile devices.
- **Reliability:** order and inventory state changes should be consistent and recoverable.
- **Security:** authentication and role-based authorization should protect administrative functions.
- **Maintainability:** the backend should use modular components with clear interfaces.
- **Energy efficiency:** the UI should avoid unnecessary animations, excessive polling and resource-heavy media.

8 UML and System Modelling

The project will produce the following models:

- **Use-case diagram:** customer, agent and administrator interactions.
- **Activity diagram:** battery ordering and installation flow.
- **Sequence diagram:** order creation, inventory validation and agent assignment.
- **Class diagram:** Customer, Order, Battery, InventoryItem, Agent, Installation and Buyback entities.
- **State diagram:** order lifecycle and exceptional states.
- **ER diagram:** relational data model for users, products, inventory, orders, payments and service records.
- **Data Flow Diagram:** movement of information between customers, system modules, administrators and service agents.

9 Software Architecture

A three-tier architecture with MVC principles is proposed.

- **Presentation layer:** responsive web interface for customers and staff.
- **Application layer:** REST APIs and business logic for authentication, ordering, inventory, dispatch and buyback.
- **Data layer:** relational database containing users, products, inventory, orders, status history and service records.

The design will initially use a modular monolithic architecture rather than microservices. This keeps the educational project manageable while retaining clear module boundaries. The design can document how individual modules could later be separated if operational scale requires it.

10 Green Software Engineering

Green software is explicitly relevant to Battify because the course includes green coding, green architecture and energy-aware UI/UX.

The project will incorporate:

- lightweight pages and optimized images;
- reduced animations and unnecessary visual effects;
- pagination instead of loading entire datasets;
- indexed database queries for frequently accessed data;
- caching of relatively static catalogue information;
- event-driven updates where practical instead of aggressive polling;
- efficient API payloads;
- dark-mode support and reduced-power UI patterns;
- monitoring of unnecessary CPU, memory and network consumption.

For the course report, these decisions can be evaluated as engineering trade-offs rather than simply being presented as visual features.

11 Software Design Principles

The system will demonstrate:

- **Separation of concerns:** UI, business logic and persistence remain independently organized.
- **Information hiding:** database implementation details remain behind service/repository interfaces.
- **High cohesion:** inventory operations are grouped within the inventory module, while dispatch operations remain in the dispatch module.
- **Low coupling:** modules communicate through well-defined APIs and interfaces.
- **Abstraction:** common operations such as order creation and status transitions are represented through service-level abstractions.

12 Testing and Verification

Testing will be planned alongside development.

12.1 Unit Testing

Examples include:

- buyback-credit calculation;
- total-price calculation;
- inventory availability checks;
- order-state transition validation;
- agent eligibility logic.

12.2 Integration Testing

Examples include:

- customer order → inventory reservation;
- inventory reservation → agent assignment;
- completed installation → inventory update;
- old-battery collection → buyback record.

12.3 System and Acceptance Testing

A complete customer scenario will be tested from battery selection through order confirmation, delivery, installation, buyback and completion.

Black-box test cases will validate expected user-visible behavior, while white-box testing will target important branches and business rules.

13 Agile Development Plan

The project can be organized into short sprints.

Sprint	Deliverable
Sprint 1	Requirements, stakeholder analysis, SRS outline and UI sketches.
Sprint 2	Authentication, customer catalogue and battery selection.
Sprint 3	Order creation, inventory and pricing modules.
Sprint 4	Agent assignment, delivery tracking and installation workflow.
Sprint 5	Buyback module, admin dashboard and notifications.
Sprint 6	Testing, debugging, green-software optimization and deployment.

A product backlog will contain user stories such as:

As a customer, I want to see available batteries for my vehicle so that I can select a compatible replacement.

As an operations manager, I want to see low-stock batteries so that I can replenish inventory before demand causes service failures.

As a delivery agent, I want to see my assigned installations so that I can complete deliveries efficiently.

14 Evaluation Criteria

The system will be evaluated using measurable software and operational metrics:

- percentage of successful order submissions;
- average API response time for core operations;
- inventory consistency after concurrent order attempts;
- percentage of test cases passed;
- order completion rate in the simulated workflow;
- average number of steps required to place an order;
- reduction in unnecessary network requests after green-UI optimization;
- usability feedback from pilot users.

15 Real-world Impact

Battify is intended to solve a real operational problem rather than being only a demonstration CRUD application. The platform can reduce the friction between battery failure and replacement by integrating discovery, stock verification, delivery, installation and old-battery recovery.

The buyback component also gives the system a reverse-logistics dimension. Instead of treating the old battery as waste at the customer level, the software records its collection and creates a traceable handoff to the recovery/recycling process.

The agricultural use case is particularly relevant to Punjab because the system can model seasonal inventory planning for tractor and pump batteries. This gives the project an opportunity to demonstrate demand-aware software rather than only static product ordering.

16 Project Scope

16.1 Minimum Viable Product

The MVP will contain:

- customer registration/login;
- battery catalogue;
- compatibility and requirement form;
- inventory database;
- order creation;
- buyback-credit calculation;
- administrator dashboard;
- delivery-agent assignment;
- order-status timeline;
- installation completion record;
- unit and integration test suite.

16.2 Future Extensions

Possible extensions include:

- WhatsApp-based order intake;
- route optimization for delivery agents;
- demand forecasting for seasonal agricultural batteries;
- automated compatibility recommendations;
- supplier-side inventory integration;
- predictive battery replacement reminders;
- multi-city expansion;
- dedicated mobile application.

17 Risks and Mitigations

- **Scope creep:** restrict the assessed implementation to the MVP and treat advanced features as future extensions.
- **Battery compatibility errors:** require structured vehicle, application and battery specifications rather than relying only on free text.
- **Inventory inconsistency:** use transactions and explicit reservation states.
- **Privacy and security:** collect only required customer information and implement role-based access.
- **Complex logistics:** simulate dispatch operations initially instead of attempting a full commercial logistics network.

18 Summary

Battify is proposed as an on-demand battery delivery, installation and buyback platform that combines e-commerce, inventory management, field service and reverse logistics.

Its strongest value for UCS503 is that the project naturally covers the complete software engineering lifecycle: requirements engineering, UML modelling, SRS documentation, architecture and design, coding practices, testing, Agile development, project management and green software practices.

The existing Battify website can serve as the initial product prototype, while the UCS503 lab implementation will focus on turning the concept into a well-engineered, testable and documented software system.